

MANUAL DE USUARIO

Sistema de Gestión de Reservas y Boletos Aéreos con Apache Cassandra

Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Ingeniería en Ciencias y Sistemas

Curso: Sistemas de Bases de Datos 2

Proyecto: Primer Proyecto

Carné: 202202072

1. Introducción

Este manual describe cómo levantar el clúster, crear el esquema, instalar dependencias, cargar los datos, ejecutar las consultas, probar TTL y realizar la prueba de tolerancia a fallos.

2. Requisitos

- Windows 10 o superior.
- Docker Desktop con Docker Compose.
- Python 3.12.
- Git.
- PowerShell.

Componente	Versión utilizada
Apache Cassandra	4.1.12
Python	3.12
cassandra-driver	3.30.1
Faker	40.39.0

3. Ubicación del proyecto

```
cd "C:\Users\Admin\Documents\BD2_2S26_Proyectos_202202072\Primer Proyecto"
```

Todos los comandos siguientes se ejecutan desde la carpeta Primer Proyecto.

4. Levantar el clúster

```
docker compose up -d
docker compose ps
docker exec cassandra1 nodetool status
```

Espere hasta observar tres nodos en estado UN. Si aparece 'No nodes present in the cluster', espere unos segundos y vuelva a ejecutar nodetool status.

5. Crear keyspace y tablas

```
Get-Content -Raw .\cql\01_keyspace.cql | docker exec -i cassandra1 cqlsh
Get-Content -Raw .\cql\02_schema.cql | docker exec -i cassandra1 cqlsh
```

Verifique el keyspace:

```
docker exec cassandra1 cqlsh -e "SELECT keyspace_name, replication FROM system_schema.keyspaces WHERE
keyspace_name = 'aerolinea';"
```

Verifique las 11 tablas:

```
docker exec cassandra1 cqlsh -e "SELECT table_name FROM system_schema.tables WHERE keyspace_name =
'aerolinea';"
```

6. Preparar Python

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

En Python 3.12 la dependencia pyasyncore está incluida en requirements.txt y permite que cassandra-driver utilice el mecanismo de conexión esperado en este entorno.

7. Verificar conexión

```
.\.venv\Scripts\python.exe scripts\01_verificar_conexion.py
```

El resultado correcto muestra conexión establecida, keyspace aerolinea, Cassandra 4.1.12 y 11 tablas.

8. Carga de prueba

```
.\.venv\Scripts\python.exe scripts\02_carga_prueba.py
```

La carga de prueba crea 1,000 reservas y sirve para validar la integración antes de la carga final.

9. Carga definitiva

```
.\.venv\Scripts\python.exe -W ignore::DeprecationWarning -u scripts\03_carga_masiva.py --reservas 100000
```

El script limpia las tablas antes de cargar. No debe ejecutarse nuevamente si se desea conservar la carga actual.

Dato esperado	Cantidad
Reservas	100,000
Pagos	100,000
Pasajeros	5,000
Vuelos	1,000
Asientos	120,000
Batch Writes	20,000

10. Consultas principales

Las consultas están en cql/03_queries.cql. Ninguna utiliza JOIN ni ALLOW FILTERING.

10.1 Q1 - Disponibilidad

```
SELECT clase, disponibles, ocupados
FROM aerolinea.disponibilidad_asientos_por_vuelo_clase
WHERE vuelo_id = 7738e6e4-6f78-5a6a-9adc-1f33c41fe130;
```

10.2 Q2 - Historial

```
SELECT fecha_salida, codigo_vuelo, aeropuerto_origen,
       aeropuerto_destino, numero_asiento, clase,
       estado_reserva, estado_pago, monto_pago
FROM aerolinea.historial_reservas_por_pasajero
WHERE pasajero_id = af65cd22-ae00-5a4a-8268-45d22029b352
   AND fecha_salida >= '2026-01-10T00:00:00Z'
   AND fecha_salida <= '2026-01-26T23:59:59Z';
```

10.3 Q3 - Manifiesto

```
SELECT numero_asiento, clase, nombre_pasajero,
       documento_identificacion, estado_reserva, estado_pago
```

```
FROM aerolinea.manifiesto_por_vuelo
WHERE vuelo_id = 7738e6e4-6f78-5a6a-9adc-1f33c41fe130;
```

10.4 Q4 - Ocupación

```
SELECT SUM(reservas_confirmadas) AS reservas_confirmadas,
       SUM(capacidad_total) AS capacidad_total,
       (CAST(SUM(reservas_confirmadas) AS decimal) * 100)
       / SUM(capacidad_total) AS porcentaje_ocupacion
FROM aerolinea.ocupacion_por_ruta_mes
WHERE origen = 'GUA'
      AND destino = 'MEX'
      AND anio_mes = '2026-01'
      AND fecha_salida >= '2026-01-01T00:00:00Z'
      AND fecha_salida <= '2026-01-31T23:59:59Z';
```

10.5 Q5 - Top N

```
SELECT codigo_vuelo, fecha_salida, origen, destino, ingreso_total
FROM aerolinea.ranking_ingresos_por_periodo
WHERE fecha_inicio = '2026-01-01'
      AND fecha_fin = '2026-01-31'
ORDER BY ingreso_total DESC
LIMIT 10;
```

11. Prueba de TTL

cql/03_queries.cql contiene una reserva temporal con USING TTL 30. Después de insertarla, TTL(estado) disminuye hasta que Cassandra elimina automáticamente el registro.

```
SELECT reserva_id, numero_asiento, estado,
       TTL(estado) AS ttl_restante
FROM aerolinea.reservas_por_id
WHERE reserva_id = 99999999-9999-4999-8999-999999999999;
```

12. Tolerancia a fallos

12.1 Tres nodos activos

```
.\.venv\Scripts\python.exe -u scripts\04_prueba_tolerancia.py --escenario 3_nodos
```

ONE, QUORUM y ALL deben responder.

12.2 Detener un nodo

```
docker stop cassandra3
docker exec cassandra1 nodetool status aerolinea
```

cassandra3 debe aparecer DN.

12.3 Probar con un nodo caído

```
.\.venv\Scripts\python.exe -u scripts\04_prueba_tolerancia.py --escenario 1_nodo_caido
```

Con RF=3, ONE y QUORUM deben responder; ALL debe fallar porque requiere tres réplicas.

12.4 Recuperar el nodo

```
docker start cassandra3
docker exec cassandra1 nodetool status aerolinea
```

Espere hasta que los tres nodos vuelvan a UN.

```
.\.venv\Scripts\python.exe -u scripts\04_prueba_tolerancia.py --escenario cluster_recuperado
```

Tras la recuperación, ALL debe volver a responder.

13. Detener y reiniciar el entorno

```
docker compose stop
docker compose start
```

Los volúmenes conservan los datos. `docker compose down` elimina contenedores y red, pero conserva los volúmenes si no se utiliza `-v`.

No utilice `docker compose down -v` si desea conservar los datos.

14. Restauración desde cero

- Instalar Docker Desktop, Python 3.12 y Git.
- Clonar el repositorio.
- Entrar a la carpeta Primer Proyecto.
- Ejecutar `docker compose up -d`.
- Esperar tres nodos UN.
- Ejecutar `01_keyspace.cql` y `02_schema.cql`.
- Crear `.venv` e instalar `requirements.txt`.
- Ejecutar `01_verificar_conexion.py`.
- Ejecutar `03_carga_masiva.py --reservas 100000`.
- Ejecutar o consultar `cql/03_queries.cql`.

15. Problemas comunes

Cassandra aún no está listo

Si `nodetool` indica que no hay nodos presentes, espere unos segundos y repita la comprobación.

Error `asyncore` con Python 3.12

Instale las dependencias de `requirements.txt`. El archivo incluye `pyasyncore`.

ALL falla con un nodo detenido

Es el comportamiento esperado: con `RF=3`, ALL requiere las tres réplicas.

Warning durante `COUNT(*)`

Aggregation query used without partition key aparece en verificaciones administrativas sobre tablas completas. Las consultas Q1-Q5 no utilizan ese patrón.

16. Evidencias

Las evidencias de clúster, carga, consultas y tolerancia a fallos se encuentran en las carpetas evidencias/cluster, evidencias/carga, evidencias/consultas y evidencias/tolerancia_fallos.

17. Consideraciones finales

Para una entrega o demostración normal se recomienda mantener los tres nodos en estado UN. El script de carga limpia las tablas antes de ejecutarse, por lo que debe utilizarse con precaución cuando ya exista información que se desee conservar.